

REPORT

DFS (Depth-First Search)

Aim

To implement Depth-First Search (DFS) to traverse a graph by exploring each path as deeply as possible before backtracking.

Objectives

- To understand the concept of graph traversal.
- To implement DFS using a stack or recursion.
- To visit all reachable vertices systematically.
- To understand the concept of backtracking.

Requirements

- Computer/Laptop
- Programming language such as C, C++, Java, or Python
- Code editor/IDE
- Graph with vertices and edges

Theory

Depth-First Search (DFS) is a graph traversal technique that explores a graph by going as deep as possible along one path before backtracking. DFS uses a **Stack** or **Recursion** to keep track of vertices. A visited list is maintained to avoid visiting the same vertex repeatedly.

Algorithm

1. Start with the given starting vertex.
2. Mark the starting vertex as visited.
3. Push the vertex into the stack or call DFS recursively.
4. Select an unvisited adjacent vertex.
5. Mark the selected vertex as visited.
6. Continue exploring until there are no unvisited adjacent vertices.
7. Backtrack to the previous vertex.

8. Repeat the process for remaining unvisited vertices.
9. Stop when all reachable vertices are visited.
10. Display the vertices in the order they were visited.

Program

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}  
  
visited = set()  
  
def dfs(node):  
    if node not in visited:  
        print(node, end=" ")  
        visited.add(node)  
        for neighbour in graph[node]:  
            dfs(neighbour)  
  
dfs('A')
```

OUTPUT

Result

Thus, the **Depth-First Search (DFS)** traversal of the given graph is successfully performed by exploring the vertices deeply and backtracking when necessary.